Mount Kenya University

STUDENT DETAILS

Name: Job Roman Barasa

Admission Number:BSCCS/2024/42922

Course: Bachelor of Science in Computer Science

Unit Code: BIT 4139

Unit Name: Ethical Hacking and Counter Measures

Lecturer Name: Mr Michael Nyoro

Academic Year: 4.1

PROJECT DETAILS

Project Title: Web Application Security and OWASP Top 10

Selected Technologies

Brave browser developer tools

OWASP Juice Shop

Nikto

GithubLink: link

Week 7: Web Application Security and OWASP Top 10

Class Demonstration 1: HTTP Request Analysis

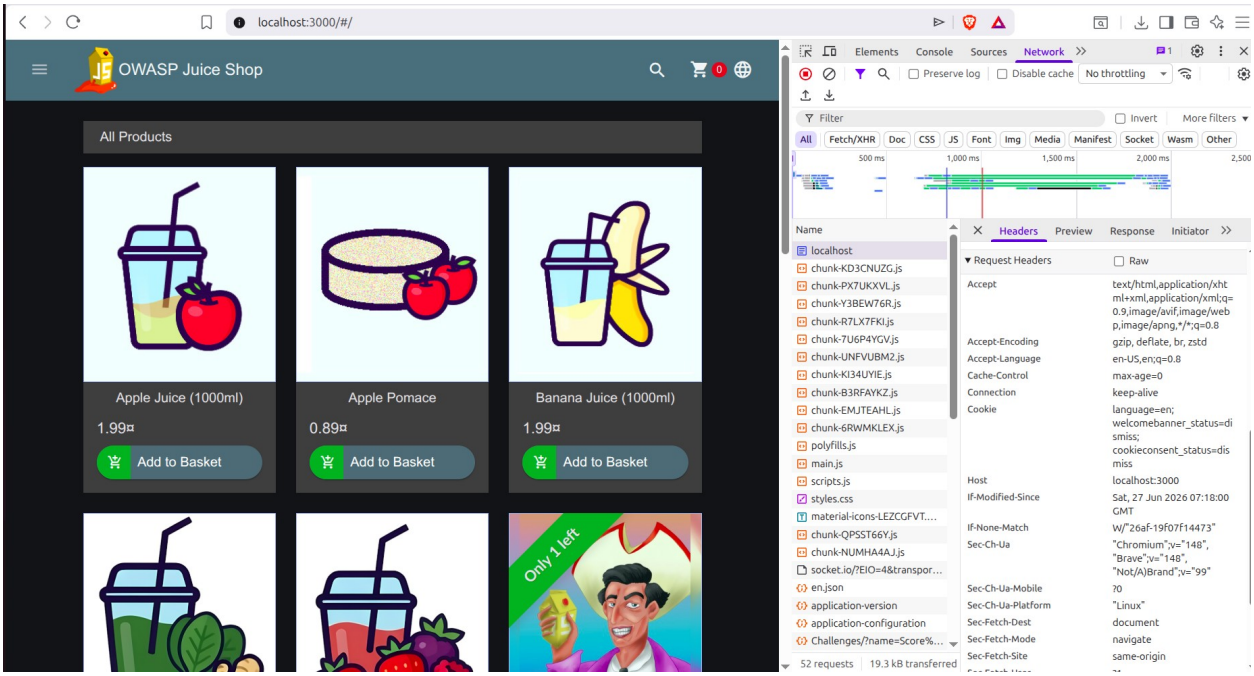


Figure 1: Request Headers

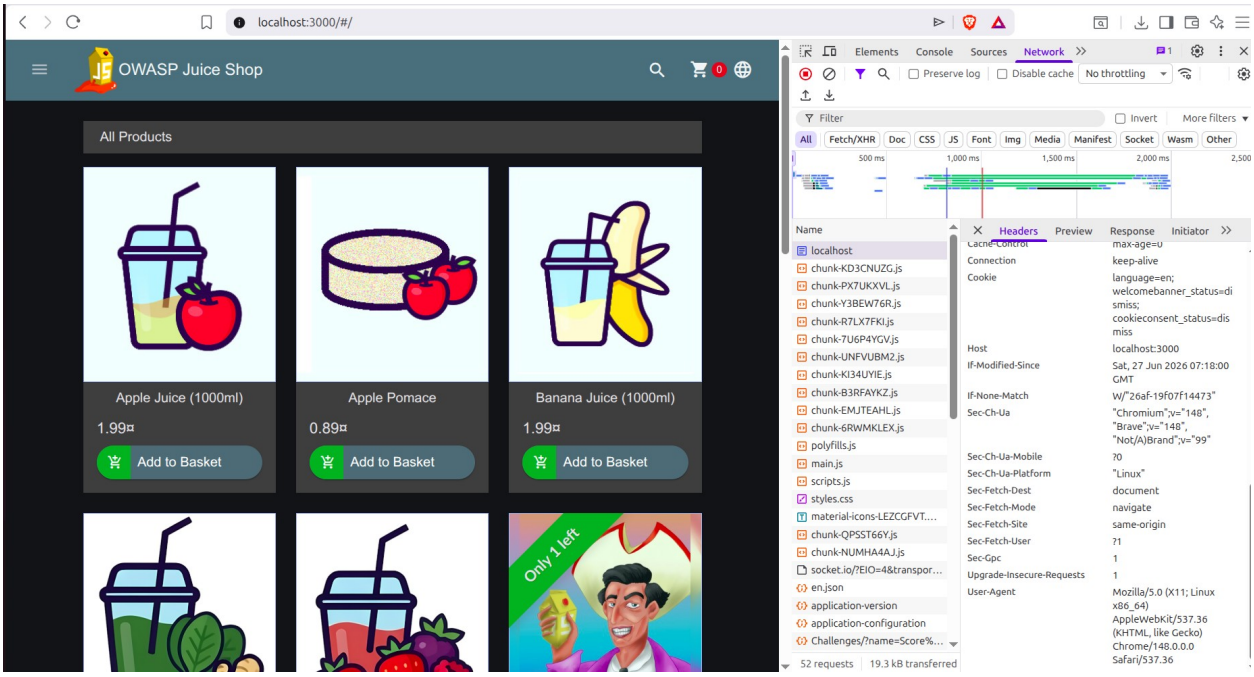


Figure 2: Request Headers 2

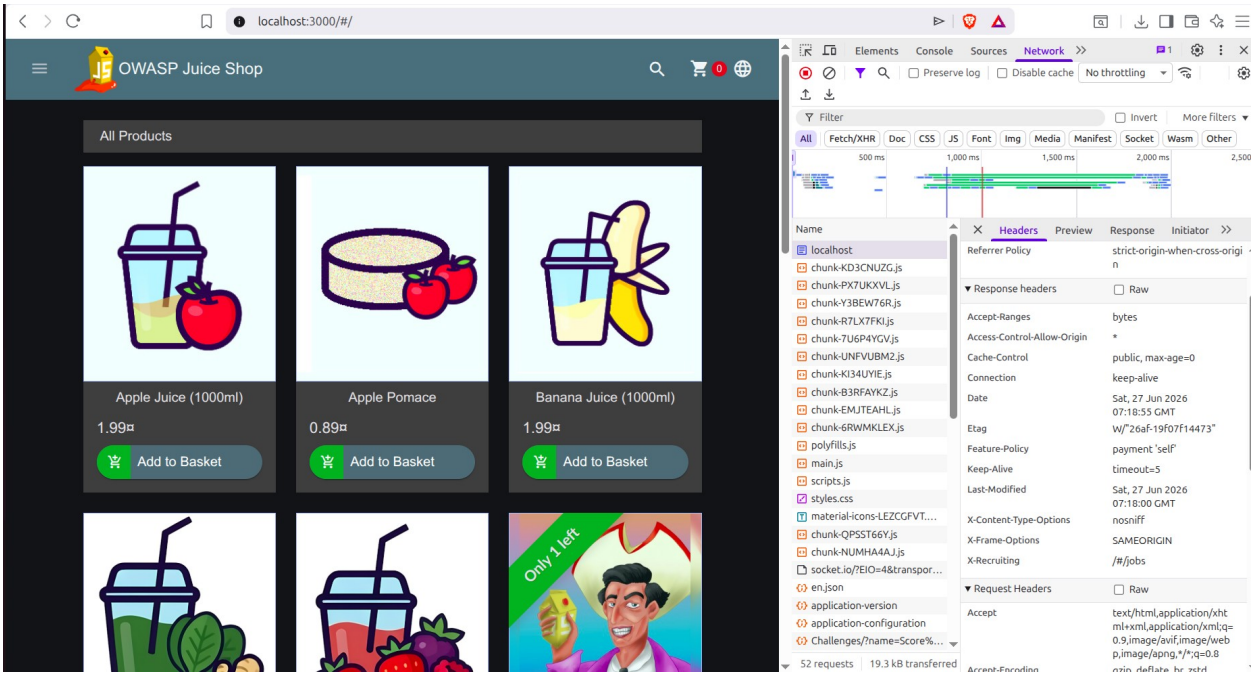


Figure 3: Response Headers

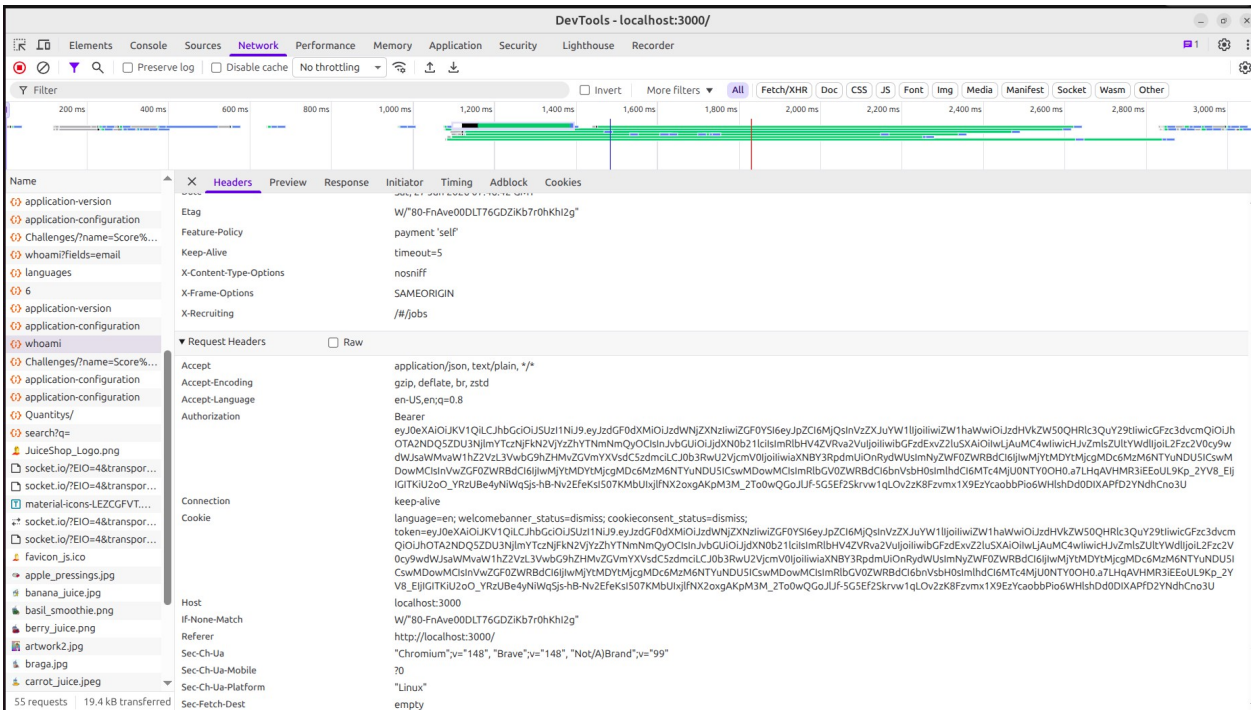


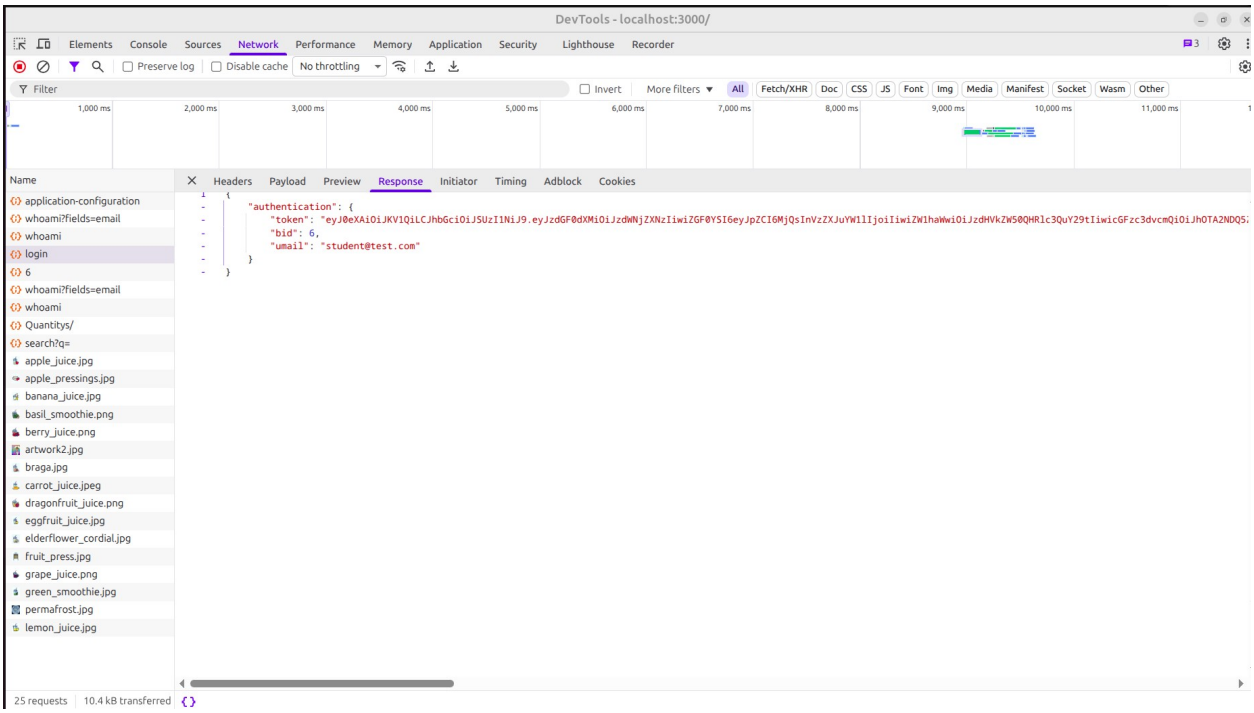
Figure 4: Authorization Mechanisms

Class Demonstration 2: Input Validation Testing

Test	Input	Result	Security Observation
Normal Search	apple	Products displayed	Search functionality worked correctly.
Long Search	100+ a characters	No results found	The application handled the long input without crashing or generating an error, indicating good input handling.
Special Characters	!@#$%^&*()...	No results found	The application safely processed special characters without exposing errors or abnormal behavior.
Invalid Email	hello	Rejected	The form required a properly formatted email address containing an @ symbol, demonstrating client-side validation.
Password Policy	6+ character password	Accepted	The application enforced a minimum password length of six characters. Additional complexity (uppercase, numbers, symbols) was not required but optional

Class Demonstration 3: Cookie Security Analysis

During the login process, the application returned a JSON Web Token (JWT) after successful authentication. Network analysis showed that authentication is token-based rather than relying solely on traditional session cookies. The response also included security headers such as X-Content-Type-Options: nosniff and X-Frame-Options: SAMEORIGIN, which help protect against MIME-sniffing and clickjacking attacks.



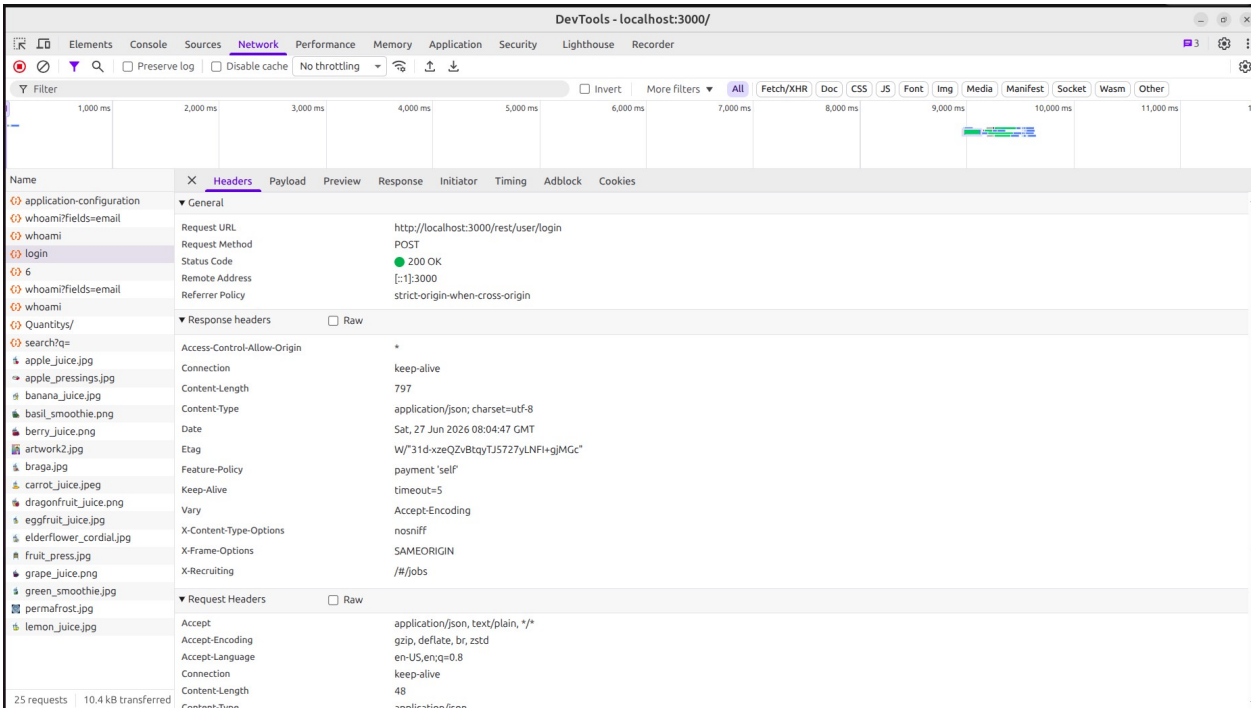


Figure 5: Login request analysis

Class Demonstration 4: Web Application Mapping

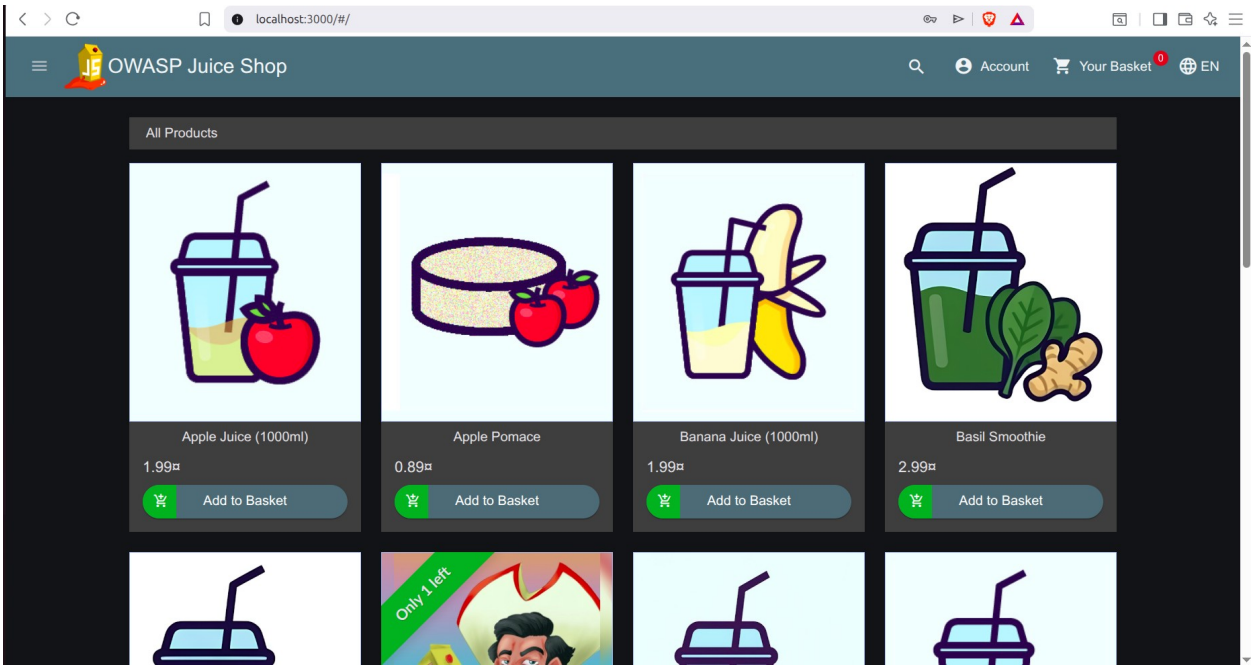


Figure 6: Home Page

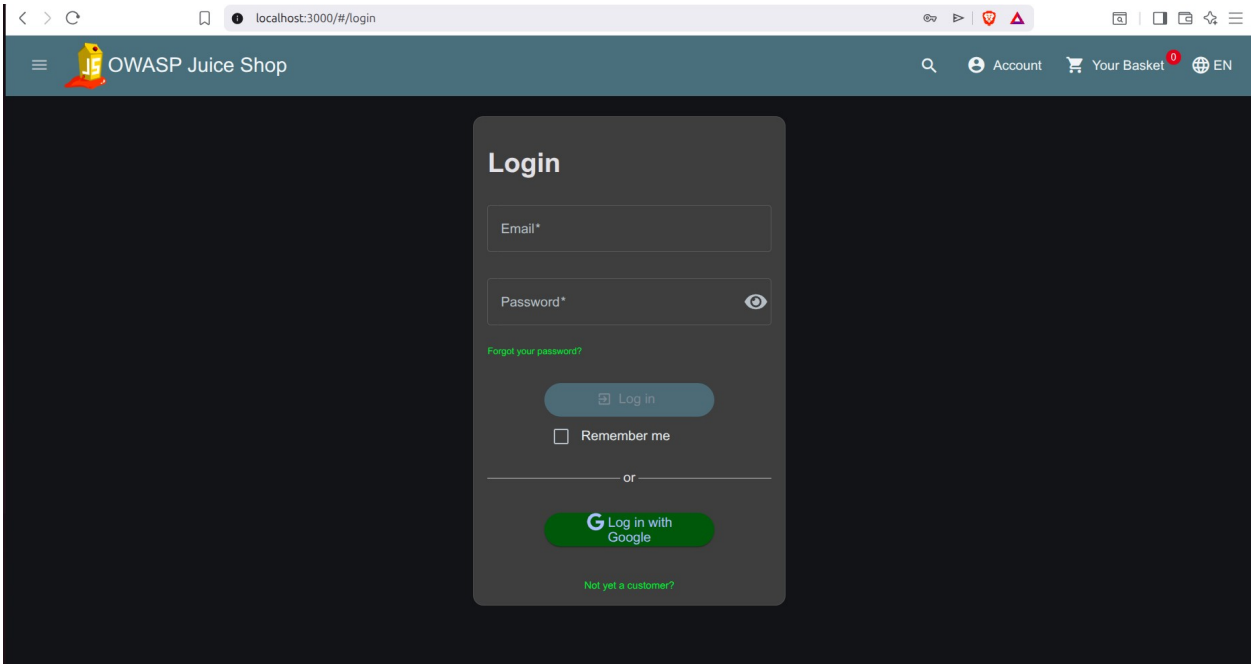


Figure 7: Login Page

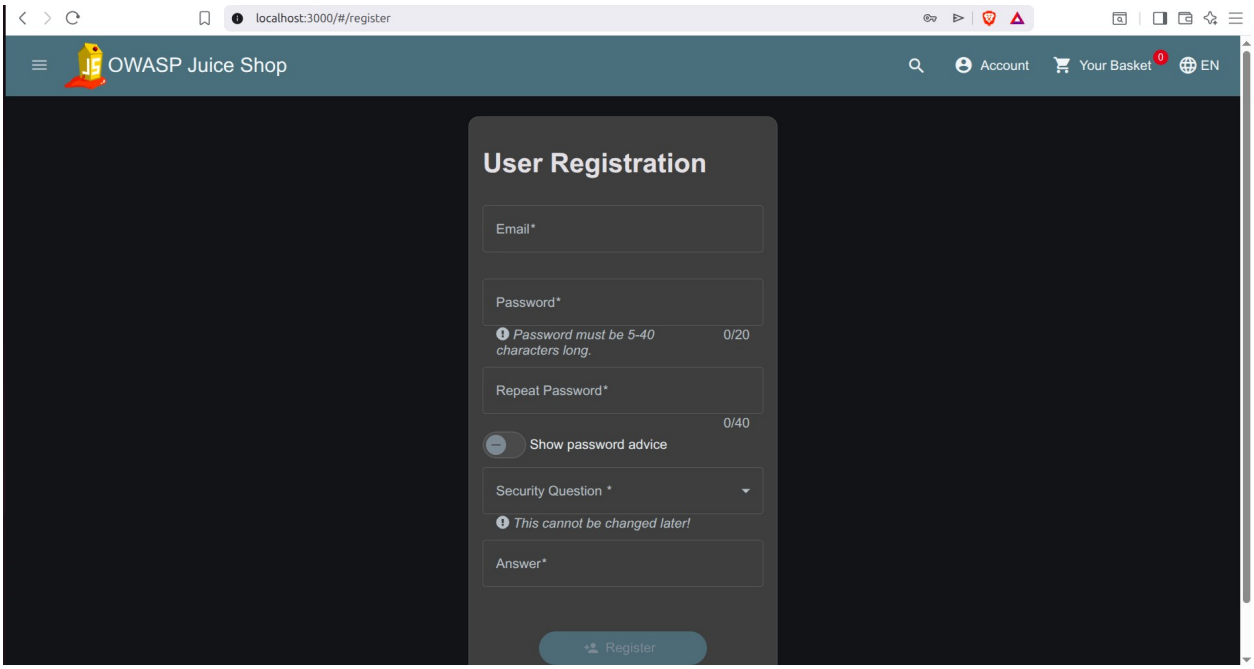


Figure 8: Registration Page

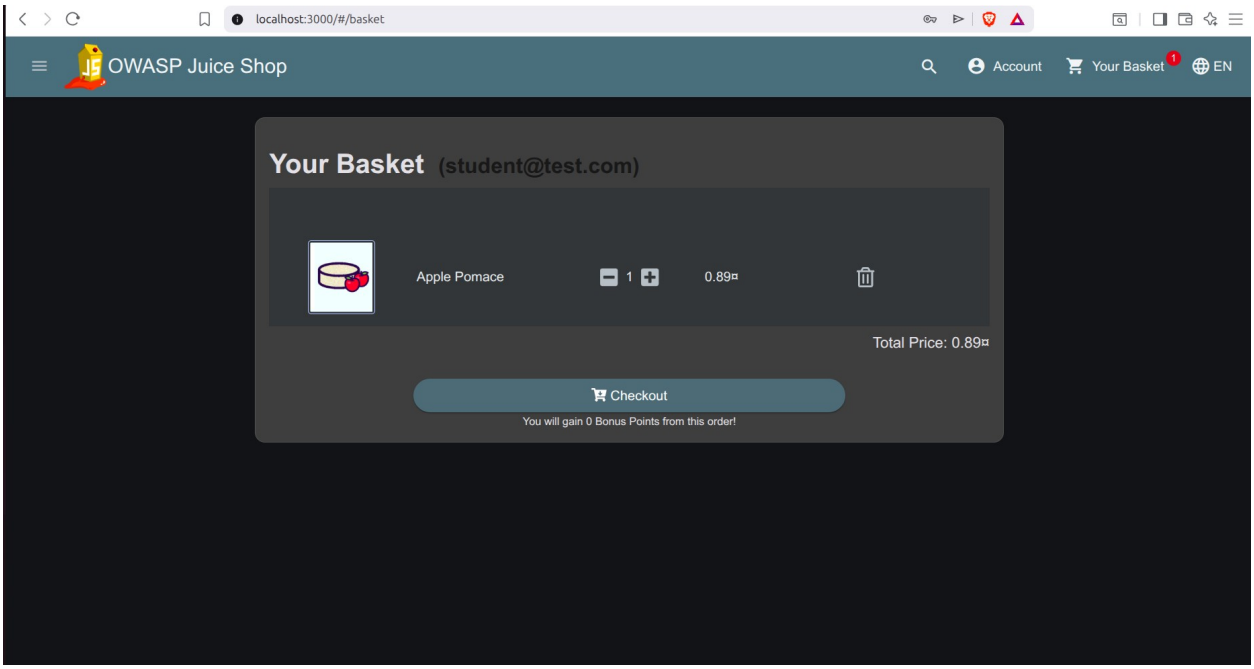


Figure 9: Basket

Class Activity 1: Vulnerability Identification Exercise

1. Authentication Findings

Strengths

- Email-based authentication.
- Registration process available.
- JWT token used after successful login.
- Authentication token successfully generated.

Weaknesses

- Minimum password length is only 5 characters
- No visible Multi-Factor Authentication
- No obvious account lockout after repeated failed logins.
- No visible CAPTCHA on the login page.

Risk Level: Medium

2. Input Validation

Tested:

- Long input
- Special characters
- Invalid email
- Password requirements

Results

The application:

- Handled long strings safely.
- Rejected invalid email formats.
- Did not crash when receiving special characters.
- Returned normal "No results found" messages.

Risk Level: Low

This suggests the application performs basic client-side and server-side validation.

3. Configuration Review

From the HTTP headers I observed:

Good security headers:

- X-Content-Type-Options: nosniff
- X-Frame-Options: SAMEORIGIN

Potential concern:

- Access-Control-Allow-Origin: *

This header allows requests from any origin. Whether this is acceptable depends on the application's intended design. For a deliberately vulnerable training application like Juice Shop, this may be intentional. In a production system, it should be carefully reviewed to ensure sensitive APIs are not unnecessarily exposed.

Risk Level: Medium

Class Activity 2: Risk Classification

Issue	Risk	Justification
Weak Password Policy	Medium	Five character passwords are easier to brute-force than longer, more complex passwords.
Missing MFA	Medium	User accounts rely only on passwords, increasing the impact of stolen credentials.
Missing Account Lockout	Medium	Unlimited login attempts can increase the risk of brute-force attacks.
Wildcard CORS (Access-Control-Allow-	Mediu	Could expose APIs to

Issue	Risk	Justification
Origin: *)	m	unintended cross-origin requests if not carefully controlled.
Input Validation	Low	The application safely handled invalid input during testing.
Email Validation	Low	Invalid email addresses were correctly rejected.
Security Headers Present	Low	The observed headers improve protection against common browser-based attacks.

Class Activity 3: Secure Design Discussion

1. Why should security be built into applications?

Security should be considered from the beginning of the software development lifecycle because fixing vulnerabilities during design and development is easier and less expensive than addressing them after deployment. Early integration also reduces the risk of data breaches and security incidents.

2. What is the cost of fixing vulnerabilities late?

Late fixes often require code redesign, emergency patches, downtime, additional testing, and can lead to financial losses, reputational damage, and legal consequences if attackers exploit the vulnerabilities before they are resolved.

3. What is the role of developers in cybersecurity?

Developers play a key role by writing secure code, validating user input, implementing proper authentication and authorization, applying security updates, and following secure coding practices to reduce vulnerabilities in applications.

Practical Task 1: OWASP Top 10 Research

OWASP Vulnerability	Definition	Example	Impact	Prevention
Broken Access Control	Occurs when users can access resources or perform actions beyond their permissions.	A normal user accesses an administrator page by changing the URL.	Unauthorized access, privilege escalation, and data theft.	Implement role-based access control, authorization checks and the principle of least privilege.
Cryptographic Failures	Sensitive information is not adequately protected through encryption.	Passwords stored in plain text or data transmitted over HTTP.	Data breaches, identity theft, and loss of confidentiality.	Use strong encryption, HTTPS, secure key management, and hash passwords with modern algorithms.
Injection	Untrusted user input is interpreted as commands by a database or operating system.	SQL Injection in a login form.	Database compromise, data loss, and unauthorized access.	Use prepared statements and input validation.
Insecure Design	Security	An online	Business logic	Apply secure-by-

OWASP Vulnerability	Definition	Example	Impact	Prevention
	weaknesses caused by poor application design rather than coding mistakes.	banking system without transaction approval for large transfers.	abuse, fraud, and unauthorized actions.	design principles, threat modeling, and security reviews.
Security Misconfiguration	Incorrect or insecure system configurations expose vulnerabilities.	Default passwords or exposed administrative interfaces.	System compromise and information disclosure.	Remove default settings, harden configurations, and perform regular security audits.
Vulnerable and Outdated Components	Using software components with known security vulnerabilities.	Running an outdated content management system or library.	Exploitation of known flaws leading to system compromise.	Regularly update software, apply security patches, and monitor dependencies.
Identification and Authentication Failures	Weak authentication or session management allows attackers to impersonate users.	Weak passwords or session hijacking.	Account takeover and unauthorized access.	Enforce strong password policies, enable MFA, and secure session management.
Software and Data Integrity Failures	Failure to verify the integrity of	Installing unsigned software	Malware infection and supply chain	Use code signing, integrity verification, and

OWASP Vulnerability	Definition	Example	Impact	Prevention
	software or updates.	updates or compromised third-party libraries.	attacks.	secure CI/CD pipelines.
Security Logging and Monitoring Failures	Insufficient logging and monitoring delay the detection of attacks.	Failed login attempts are not recorded or monitored.	Undetected attacks and delayed incident response.	Implement centralized logging, real-time monitoring, and incident response procedures.
Server-Side Request Forgery (SSRF)	A server is tricked into making requests to unintended internal or external resources.	A web application fetches a URL supplied by a user without validation.	Exposure of internal systems, data leakage, and cloud service compromise.	Validate user-supplied URLs, restrict outbound requests, and segment internal networks.

Practical Task 2: Web Application Assessment

Target Application

	Details
Application	OWASP Juice Shop
URL	http://localhost:3000

Details	
Testing Environment	Ubuntu 24.04
Tools Used	Brave Developer Tools, OWASP ZAP, Nikto

Identified Forms

Form	Purpose	Observation
Login	User authentication	Requires email and password
Registration	New account creation	Validates email format and password length
Search	Product search	Takes user input and returns matching products
Feedback	Customer feedback	Allows users to submit comments
Product Review	Product ratings and reviews	Available after login
Password Reset	Account recovery	Uses security question and answer

Authentication Mechanisms

Users authenticate using an email address and password.

Successful login returns a JSON Web Token (JWT).

The token is used to maintain the authenticated session.

No Multi-Factor Authentication (MFA) was observed.

No visible account lockout mechanism was observed.

User Roles

Guest- Can browse products without logging in

Registered User- Can purchase products, leave reviews and manage their account

Administrator- Not accessible to normal users

Input Validation Findings

Long Input- Application remained stable and returned "No results found."

Special Characters- Safely handled without errors

Invalid Email- Rejected until a valid email format was entered

Password- Minimum length of five characters enforced

Basic input validation is implemented and no abnormal behavior was observed during testing.

Potential Vulnerabilities

Observation	Risk	Recommendation
Minimum password length only	Medium	Require stronger passwords (8–12+ characters with complexity rules).
No visible MFA	Medium	Implement multi-factor authentication for enhanced account security.
No visible account lockout	Medium	Introduce temporary account lockout or rate limiting after repeated failed login attempts.
Wildcard CORS (Access-Control-Allow-Origin: *)	Medium	Restrict allowed origins where appropriate in production environments.

Practical Task 3: Security Improvement Proposal

Title: Security Improvement Proposal for OWASP Juice Shop

1. Introduction

This proposal presents recommendations to improve the security of the OWASP Juice Shop web application. The recommendations are based on observations made during the web application assessment using browser developer tools, OWASP ZAP, and Nikto. The objective is to reduce security risks, strengthen user account protection and improve the application's overall security posture.

2. Identified Security Issues

The following issues were identified during the assessment:

Issue	Risk Level
Weak password policy	Medium
No Multi-Factor Authentication	Medium
No visible account lockout after repeated failed logins	Medium
Cross-Origin Resource Sharing configured with Access-Control-Allow-Origin: *	Medium

3. Proposed Security Improvements

Recommendation 1: Strengthen the Password Policy

Current Observation

The application accepts passwords with a minimum length of six characters.

Recommendation

Require users to create passwords that:

- Are at least 8–12 characters long.
- Include uppercase and lowercase letters.
- Include numbers.
- Include special characters.

Benefit

Stronger passwords reduce the risk of brute-force and dictionary attacks.

Recommendation 2: Implement Multi-Factor Authentication

Current Observation

Authentication relies solely on an email address and password.

Recommendation

Implement MFA using:

- Authentication apps (recommended), or
- Email verification codes, or
- SMS verification where appropriate.

Benefit

Even if a password is compromised, unauthorized users cannot easily access the account.

Recommendation 3: Implement Account Lockout and Rate Limiting

Current Observation

No account lockout mechanism was observed during testing.

Recommendation

- Lock an account temporarily after several failed login attempts.
- Apply rate limiting to login requests.

Benefit

This helps prevent automated brute-force attacks.

Recommendation 4: Restrict Cross-Origin Resource Sharing (CORS)

Current Observation

The response header:

Access-Control-Allow-Origin: *

was observed during testing.

Recommendation

Restrict allowed origins to trusted domains instead of allowing all origins.

Benefit

Reduces the risk of unauthorized websites interacting with the application's resources.

Recommendation 5: Improve Logging and Monitoring

Recommendation

Implement centralized logging to record:

- Failed login attempts
- Administrative actions
- Authentication events
- Suspicious requests

Configure alerts for unusual activities.

Benefit

Improves incident detection and supports forensic investigations.

Recommendation 6: Regular Security Testing

Recommendation

Perform regular:

- Vulnerability assessments
- Penetration testing
- Security code reviews
- Dependency updates

using tools such as:

- OWASP ZAP
- Nikto
- Nmap

Benefit

Helps identify and remediate vulnerabilities before they can be exploited.

4. Expected Benefits

Implementing these recommendations will:

- Improve user account protection.
- Reduce the likelihood of unauthorized access.
- Strengthen authentication mechanisms.
- Improve compliance with secure development practices.
- Reduce exposure to common web application attacks.

5. Conclusion

The assessment demonstrated that OWASP Juice Shop includes several important security features, including JWT-based authentication, input validation, and protective HTTP security headers. However, improvements such as stronger password policies, multi-factor authentication, account lockout mechanisms, improved logging, would further strengthen the application's security and reduce potential risks.

Challenge Task: Secure Web Application Design Framework.